Seminar: Automated Mobile Application Testing

PAULINA PAZDZIERZ, Technical University of Munich, Germany

Usability and accessibility play a central role in the success of mobile applications, but traditional evaluation methods are costly and difficult to scale. To address these limitations, recent research explores the use of large language models (LLMs) for automated usability and accessibility testing. This paper reviews and analyzes existing LLM-based approaches for evaluating mobile applications, focusing on how these models are used, what inputs they rely on, and how their performance is assessed. The reviewed studies show that LLMs can support usability evaluation and accessibility auditing, particularly for screen-level usability issues and interaction-dependent accessibility problems. However, the results also indicate that LLM-based approaches do not replace manual evaluation and are best used as complementary tools.

Additional Key Words and Phrases: Mobile applications; usability testing; accessibility evaluation; large language models; automation

1 Introduction

With the widespread development of technology and increased internet penetration, mobile applications have become an important part of everyday life [8]. The increasingly common ownership of smartphones has created opportunities for completely new kinds of software, as well as for software that provides the same functionality as websites but is often preferred by users due to quicker access and easier use [7]. Because of these advantages, mobile applications have become very popular, and a large number of new apps are continuously being created. According to Grand View Research, the global mobile application market size was estimated at 252.89 billion USD in 2023 and is expected to grow at a compound annual rate of 14.3% until 2030 [3].

Despite this rapid growth, studies show that only about 0.5% of applications succeed in the marketplace [16]. One of the most frequently reported reasons for application failure is poor user experience [8]. User experience (UX) is a broad concept, but two of its key components are usability and user acceptance, which directly influence whether users continue to use an application or abandon it [8]. Ensuring high usability is therefore a crucial requirement for mobile applications.

Usability is commonly defined as "a concept that essentially refers to how easy it is for users to learn a system, how efficient they can be once they have learned it, and how enjoyable it is to use it" [11]. In practice, usability affects many aspects of interaction, such as whether users can easily understand how to complete tasks, whether interface elements are clearly labeled, and whether feedback is provided in an understandable way. Closely related to usability is accessibility, which refers to whether a product or service can be used by everyone, including users with disabilities, regardless of how they interact with the system [6]. In mobile applications, accessibility is particularly important due to small screen sizes, touch-based interaction, and the frequent use of assistive technologies such as screen readers [6].

To ensure both usability and accessibility, mobile applications must be systematically evaluated. Traditional usability and accessibility testing of mobile applications is typically conducted in two main ways. The first approach involves testing with real users, who are asked to perform specific tasks while their interactions are observed. During such studies, evaluators can identify problems such as unclear navigation, confusing labels, overwhelming layouts, or difficulties encountered when using assistive technologies. The second common approach is expert-based evaluation, where usability or accessibility experts inspect the interface and assess it based on their knowledge of design principles and guidelines. Both methods provide a realistic assessment of the user interface, as they are grounded in human judgment and real interaction context.

Author's Contact Information: Paulina Pazdzierz, ge48yog@tum.de, Technical University of Munich, Germany.

In addition to these evaluation methods, usability and accessibility considerations are often addressed during development by following established design guidelines, such as appropriate spacing of interface elements, sufficient text contrast, and intuitive navigation structures. While these practices help prevent many issues, they do not eliminate the need for systematic testing, as real user interaction often reveals problems that are not obvious during design.

However, manual evaluation methods have significant limitations. Conducting usability tests across many scenarios and user groups is time-consuming, and the cost of manual testing is high due to the need to recruit participants or pay experts. In practice, it is rarely possible to test all possible interaction paths or application states, resulting in limited coverage. For example, even a small usability study involving five participants can cost between 10,000 and 50,000 USD and require between 11 and 27 hours of testing time [10, 12]. These constraints make frequent and large-scale evaluation difficult, especially for rapidly evolving mobile applications.

As a result, there is a growing need for automation in usability and accessibility testing [4]. Automated UI testing generally refers to techniques that evaluate applications without direct human involvement, often by checking predefined rules or measurable properties. Common automated tools for usability and accessibility focus on aspects such as the presence of labels, contrast ratios, or touch target sizes. While these tools are efficient and easy to apply, they are largely limited to predefined and unambiguous checks. They lack a human-like perspective and often fail to capture issues that depend on context, semantics, or user expectations. For example, an automated tool may verify that a button has a label, but it cannot assess whether placing a “Confirm” button next to a “Cancel” button may confuse users.

User experience is inherently subjective, and not all usability or accessibility questions can be reduced to simple yes-or-no checks. Although standards and guidelines for mobile interface design exist, they often describe high-level principles and desired user experiences rather than concrete, fully measurable rules. Many usability guidelines, such as Jakob Nielsen’s usability heuristics, require interpretation and judgment rather than direct rule enforcement [11]. As a result, traditional automated tools struggle to evaluate these aspects effectively.

This creates a need for automated testing approaches that can incorporate a more human-like understanding of user interfaces and interaction context. Large language models (LLMs), which are capable of processing natural language, reasoning about context, and integrating information from multiple sources, offer a promising direction in this regard. This report reviews and analyzes existing approaches that use LLMs for automated usability and accessibility testing of mobile applications.

2 Literature Review and Background

This section reviews existing approaches to usability and accessibility evaluation and explains how recent work using large language models builds on these methods and addresses some of their limitations.

2.1 Manual Usability and Accessibility Evaluation

Manual evaluation is widely regarded as the most reliable way to assess usability and accessibility of mobile applications [4, 17, 19, 21]. In usability testing with real users, participants are typically asked to complete predefined tasks while their interactions are observed. Such studies often reveal issues that are difficult to anticipate during design.

Accessibility evaluation is similarly conducted through manual testing, often involving users with disabilities or accessibility experts interacting with applications using assistive technologies such as screen readers. Several reviewed papers treat findings from manual testing as the reference standard against which automated approaches are compared [17, 19, 21].

At the same time, the literature highlights the limitations of manual testing, particularly its high cost and limited scalability [4, 17, 19, 21].

2.2 Expert-Based Evaluation

Expert-based evaluation relies on trained specialists inspecting interfaces based on usability heuristics or accessibility guidelines. Several reviewed papers attempt to automate expert-style inspection using LLMs [15, 18, 21]. Although expert-based evaluation is generally less costly than user testing [11], it still requires significant human effort and does not scale well to frequent design iterations or large numbers of screens [15].

2.3 Traditional Automated Usability and Accessibility Tools

Traditional automated tools reduce evaluation effort by checking predefined rules such as accessibility labels or contrast ratios [1, 19]. While efficient, these tools are limited in their ability to capture contextual and interaction-dependent issues. Studies report that rule-based tools often fail to detect problems related to screen reader behavior or confusing interface semantics [1, 18, 19].

2.4 Motivation for LLM-Based Evaluation

The limitations of manual testing, expert evaluation, and rule-based automation motivate LLM-based approaches. The reviewed papers argue that LLMs offer scalability and semantic understanding that is lacking in traditional automated tools [15, 17, 21]. Importantly, all papers frame LLM-based testing as complementary to human evaluation rather than a replacement [1, 15, 17–19, 21].

3 How Large Language Models Are Used for Automated Usability and Accessibility Testing

The reviewed literature shows that large language models are used in several distinct ways for automated usability and accessibility testing of mobile applications. While individual systems differ in their technical implementation, common patterns emerge with respect to how the user interface is analyzed, what information is provided to the model, and what role the LLM plays in the evaluation process. One of the most important distinctions across the reviewed papers is whether the evaluation is conducted statically, without executing the application, or dynamically, based on interaction with the running app.

3.1 Static and Dynamic LLM-Based Evaluation Approaches

Static approaches analyze representations of the user interface without executing the application or observing real user interaction. In these systems, the LLM is typically provided with UI screenshots, structured UI descriptions derived from code or design tools, or mockups created during the design phase, and asked to reason about usability or accessibility issues based on this information.

A representative example is Synthetic Heuristic Evaluation [21], where the LLM performs a heuristic inspection of mobile UI screenshots. The model is prompted with usability heuristics and asked to identify potential violations in a way similar to a human expert. The authors show that this approach is effective at identifying issues such as unclear labels, inconsistent layouts, or missing feedback that are visible directly on the screen. At the same time, they explicitly note that issues depending on multi-step interaction or navigation history are often missed, as the analysis is limited to static representations.

Another static approach is presented in Generating Automatic Feedback on UI Mockups with Large Language Models [2], where the LLM analyzes design mockups before the application is implemented. This enables early-stage usability evaluation during the design process, without

executing the application or involving users. The paper emphasizes that this approach is particularly useful during iterative design, where frequent manual evaluation would be costly or impractical.

Static analysis is also used in UISGPT: Automated Mobile UI Design Smell Detection [18]. In this work, the LLM is provided with structured descriptions of mobile UI elements and asked to detect so-called design smells, such as misleading labels or inconsistent spacing. These issues are treated as indicators of poor usability rather than functional errors. The authors highlight that such problems are difficult to capture using simple rule-based checks, but can be reasoned about by an LLM given sufficient UI context.

In contrast, dynamic approaches incorporate execution or interaction with the application into the evaluation process. Rather than analyzing the interface in isolation, these systems observe or generate interaction traces and use LLMs to reason about runtime behavior.

Dynamic evaluation is particularly common in accessibility-focused work. For example, ScreenAudit [19] analyzes screen reader transcripts generated while navigating mobile applications. The LLM reasons about the spoken output produced by the screen reader and identifies accessibility issues such as misleading announcements, incorrect focus order, or missing feedback for dynamic content. The authors demonstrate that many of these issues cannot be detected through static analysis, as they only become apparent during runtime interaction.

Another dynamic approach is presented in AXNav [14], where accessibility test instructions written in natural language are translated into interaction sequences that are replayed on the application. The LLM then evaluates whether tasks can be completed using assistive technologies. This explicitly models task execution and highlights accessibility barriers that prevent users from completing real workflows.

TaskAudit [20] presents another dynamic accessibility approach. In this work, the LLM executes predefined tasks within the application and observes whether the tasks can be completed using assistive technologies. Accessibility issues are identified when task execution fails or behaves unexpectedly. This allows the system to detect problems that only appear during multi-step interaction and would not be found through static analysis.

Dynamic usability evaluation is explored in Applying Large Language Model to User Experience Testing [4], where the LLM analyzes logs and screenshots captured during user interaction with a mobile application. The authors show that this enables detection of interaction-level usability issues, but also note that the results depend heavily on the quality and coverage of the recorded sessions.

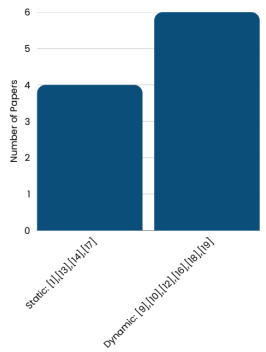

Figure 1: Distribution of static and dynamic LLM-based evaluation approaches. Only papers with clearly defined static or dynamic evaluation methods are included.

3.2 Inputs Provided to the LLM

Across both static and dynamic approaches, the reviewed papers rely on a variety of inputs to enable LLM-based evaluation. The choice of input strongly influences what aspects of usability or accessibility can be assessed.

In usability-focused static approaches, UI screenshots and visual representations are the most commonly used inputs. For example, Synthetic Heuristic Evaluation [21] and AIHeurEval [15] provide screenshots of mobile application screens, allowing the LLM to reason about layout, visual hierarchy, and labeling. Similarly, Generating Automatic Feedback on UI Mockups [2] uses design mockups as input to evaluate usability before implementation.

Several papers rely on structured UI descriptions, such as UI hierarchies or metadata derived from code. UISGPT [18] and AIHeurEval [15] use such representations to enable more precise reasoning about interface elements and their relationships. Compared to raw screenshots, these inputs reduce ambiguity and support more systematic evaluation.

Dynamic approaches frequently use interaction logs and execution traces. In Applying Large Language Model to User Experience Testing [4], logs and screenshots captured during interaction are used to evaluate whether the application provides appropriate feedback at different stages of task execution. In accessibility-focused work, screen reader transcripts play a central role. ScreenAudit [19] and AXNav [14] rely on spoken output generated during navigation to assess how the application is perceived by users relying on assistive technologies.

Reca11 [5] uses transcripts from recorded accessibility user tests as input to the LLM. The model analyzes what users say during testing and based on that creates the report about accessibility issues.

Some papers also include task descriptions, goals, or user personas as part of the input. For instance, SimUser [17] provides the LLM with user profiles and scenarios, enabling the model to generate feedback from different user perspectives. This reflects the fact that usability issues often depend on user intent rather than on interface structure alone.

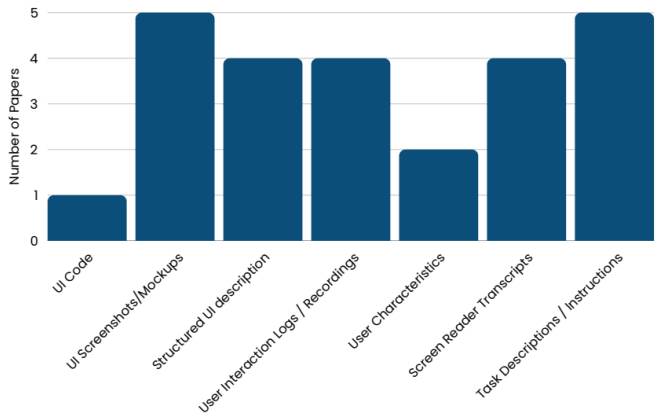

Fig. 1. Figure 2 summarizes the types of inputs provided to LLMs across the reviewed papers. UI screenshots and structured UI descriptions dominate usability-focused approaches, while accessibility-focused work relies heavily on interaction-based inputs such as screen reader transcripts and execution traces. This highlights the close relationship between input choice and evaluation scope.

3.3 What the LLM Is Asked to Evaluate

The reviewed papers also differ in what the LLM is explicitly asked to evaluate. In usability-focused work, the evaluation typically targets heuristic violations, clarity of feedback, consistency across screens, or general design quality [15, 18, 21]. In these cases, the LLM effectively acts as a proxy for a usability expert.

In accessibility-focused approaches, the evaluation targets are more specific and interaction-dependent. Papers such as ScreenAudit [19] and AXNav [14] focus on whether screen reader output is meaningful, whether focus order follows a logical sequence, and whether tasks can be completed using assistive technologies. These issues are inherently contextual and cannot be fully assessed through static rules.

Across both domains, the LLM is usually given a clear instruction or goal, such as identifying usability issues or evaluating task completion. Several papers emphasize that prompt formulation and task definition play a critical role in determining the quality of the results [4, 15, 21].

3.4 Summary of LLM Usage Patterns

Taken together, the reviewed literature shows that LLMs are primarily used as automated evaluators that approximate human judgment, rather than as tools for strict rule enforcement. Static approaches prioritize scalability and early feedback, while dynamic approaches provide richer interaction context at higher cost and complexity. The choice of inputs and evaluation targets strongly determines which types of usability and accessibility issues can be detected.

This analysis provides the foundation for understanding how these approaches are evaluated in practice, which is discussed in the following section.

4 Evaluation and Results

This section analyzes how the reviewed papers evaluate LLM-based usability and accessibility testing approaches and what results they report. The evaluation strategies vary considerably across studies, reflecting differences in goals, application types, and evaluation targets. While some papers report detailed quantitative results, others rely on qualitative assessment or expert judgment. Taken together, these evaluations provide insight into where LLM-based approaches perform well and where their limitations remain.

4.1 Evaluation Baselines and Methodologies

Across usability-focused papers, evaluation is most commonly performed by comparing LLM-generated findings against human-based references, such as usability experts or real users. For example, Synthetic Heuristic Evaluation [21], and Generating Automatic Feedback on UI Mockups with Large Language Models [2] evaluate LLM outputs by comparing them to expert-identified usability issues or expert assessments of feedback quality. In these studies, human judgment is treated as the ground truth for determining whether identified issues are valid and relevant.

In contrast, accessibility-focused papers often include comparisons against traditional rule-based tools, in addition to manual audits. ScreenAudit [19], AXNav [14], and LLMs for Accessibility in Mobile Apps: Detection and Repair [1] explicitly compare LLM-based detection to conventional accessibility checkers, highlighting differences in coverage and types of issues detected.

A smaller subset of papers also evaluates differences between models or prompting strategies. For instance, UISGPT [18] and AIHeurEval [15] analyze how different LLM configurations or prompt formulations affect the quality and consistency of detected usability issues. These comparisons are generally exploratory and aim to understand sensitivity rather than to establish definitive performance rankings.

4.2 Quantitative Results in Usability-Focused Evaluation

Among the reviewed usability-focused papers, Synthetic Heuristic Evaluation [21] reports the most detailed quantitative results. The authors evaluate LLM-based heuristic inspection on two mobile applications: a learning application and a rental application. For the learning application, human experts identified 63% of the known usability issues, while the LLM-based approach identified 77%. For the rental application, the detection rates were 57% for human experts and 73% for the LLM. These results indicate that the LLM achieved higher recall than individual experts in both cases. However, the authors explicitly caution that some LLM-identified issues were judged to be less severe or overly generic, indicating that higher detection rates do not necessarily translate into higher practical usefulness.

SimUser [17] evaluates LLM-generated usability feedback by comparing it with feedback collected from real users performing tasks in a mobile interface. The authors report overlap rates ranging from approximately 35% to nearly 100%, depending on the user persona and issue category. High overlap is observed for common usability problems, such as unclear navigation or missing feedback, while lower overlap occurs for issues that depend on individual user strategies or prior experience. This suggests that LLM-based user simulation captures general usability problems effectively but struggles with highly personalized or context-specific issues.

Other usability-focused papers focus primarily on qualitative evaluation results [9, 13]. In Generating Automatic Feedback on UI Mockups [2], expert designers assess the usefulness of LLM-generated feedback during early design stages. The results indicate that the feedback often aligns with expert concerns, particularly regarding layout and labeling, but may lack actionable detail. Similarly, UISGPT [18] reports that LLM-based detection of UI design smells produces results comparable to expert inspection for certain categories, while also generating false positives that require manual filtering.

AIHeurEval [15] reports that LLMs are effective at identifying heuristic violations across multiple screens, but it also acknowledges limitations in evaluating interaction flow and cross-screen consistency, and therefore relies on expert judgment rather than strict quantitative metrics.

4.3 Quantitative and Qualitative Results in Accessibility-Focused Evaluation

Accessibility-focused papers report evaluation results using metrics and observations that emphasize interaction-dependent issues and task-level barriers rather than heuristic coverage alone. Reca11 [5] evaluates an LLM-based system on 36 accessibility user test sessions, covering 319 accessibility issues reported by users with disabilities. The system achieves a recall of 98%, missing 6 issues, and a precision of 85%. The authors report that false positives primarily result from ambiguous spoken feedback, repeated screen reader announcements, and unclear contextual references, indicating that transcript-based analysis is effective but requires human validation.

In LLMs for Accessibility in Mobile Apps: Detection and Repair [1], the authors report lower detection rates when analyzing applications directly. Depending on the application and issue category, the LLM-based approach identifies approximately 35–40% of known accessibility issues. While this recall is lower than that reported by transcript-based approaches, many detected issues involve missing feedback or interaction behavior not captured by traditional rule-based tools. At the same time, the approach produces false positives and misses a substantial portion of violations, limiting its suitability as a standalone evaluation method.

Other work emphasizes comparative coverage rather than aggregate accuracy. ScreenAudit [19] compares LLM-based analysis against traditional automated accessibility checkers and manual audits. Prior studies cited in the paper show that existing automated tools detect only 4 out of 40 distinct accessibility problem types encountered by blind and low-vision users. ScreenAudit [19]

demonstrates that LLM-based analysis detects a broader range of issues related to screen reader announcements, focus order, and dynamic content feedback that are missed by conventional tools.

Task-level evaluation is explored in AXNav [14], where accessibility is assessed based on whether users relying on assistive technologies can complete specified tasks. Rather than counting individual violations, the approach evaluates task success and failure across multi-step workflows and reveals accessibility barriers that prevent task completion even when interfaces technically satisfy accessibility guidelines.

TaskAudit [20] reports similar findings at the task level. Instead of counting individual accessibility violations, the evaluation focuses on whether users can successfully complete tasks using assistive technologies. The results show that some tasks fail even when the interface meets accessibility guidelines, revealing barriers that are not detected by static checks.

4.4 Cross-Paper Interpretation

Considering the reviewed papers together, several consistent patterns emerge. Usability-focused studies show that LLM-based approaches can achieve coverage comparable to, and in some cases higher than, individual human evaluators for screen-level and heuristic-based issues [17, 21]. However, these approaches consistently struggle with interaction flow, cross-screen consistency, and context-dependent problems [15].

Accessibility-focused work demonstrates clearer advantages over traditional automated tools, particularly when dynamic interaction data is available [1, 19]. At the same time, moderate detection rates, false positives, and limited coverage, as well as narrow coverage of disability types, limit the applicability of these methods as standalone solutions.

Importantly, all reviewed papers, regardless of domain, explicitly position LLM-based usability and accessibility testing as complementary to manual evaluation, not as a replacement [1, 15, 17–19, 21]. The evaluation results support the use of LLMs as scalable support tools that reduce effort and increase coverage, while still requiring human oversight for validation and decision-making.

5 Limitations and Challenges

Although the reviewed papers show that large language models can support automated usability and accessibility testing, they also report several recurring limitations. These limitations appear across different approaches and are not limited to individual implementations.

A key limitation concerns interaction flow and context. Static approaches that rely on screenshots, mockups, or structured UI descriptions are limited to what is visible on individual screens. Papers such as Synthetic Heuristic Evaluation [21], AIHeurEval [15], and Generating Automatic Feedback on UI Mockups [2] explicitly note that their methods struggle to detect issues that only emerge during multi-step interaction, such as missing feedback after navigation or inconsistent behavior across screens. Even when multiple screens are analyzed, cross-screen consistency remains difficult to evaluate reliably [15].

Dynamic approaches address some of these limitations by incorporating interaction data, but they introduce new challenges. Papers such as Applying Large Language Model to User Experience Testing [4] and ScreenAudit [19] emphasize that results strongly depend on the quality and coverage of recorded interaction traces. If relevant interaction paths are not captured, corresponding usability or accessibility issues remain undetected.

Another recurring challenge is output reliability. Several papers report that LLMs generate false positives or overly generic findings that require manual filtering. In Synthetic Heuristic Evaluation [21], the LLM identifies more issues than human experts, but some of these are judged to be less relevant. Similarly, LLMs for Accessibility in Mobile Apps: Detection and Repair [1] reports false

positives and missed violations, indicating that a non-trivial portion of detected accessibility issues are false positives.

Accessibility-focused work also highlights a limited scope. Most studies concentrate on screen reader interaction and visual impairments, as seen in ScreenAudit [19] and AXNav [14]. Other accessibility dimensions, such as motor or cognitive impairments, are largely outside the scope of current LLM-based approaches.

Across all reviewed papers, a consistent conclusion is that LLM-based usability and accessibility testing does not replace human evaluation. Instead, authors emphasize that these methods are best used as complementary tools that support experts and testers by increasing coverage and reducing effort, while still requiring human judgment for validation and prioritization [1, 15, 17–19, 21].

6 Future Directions

Future work identified in the reviewed literature focuses on improving coverage, reliability, and integration into development workflows. Several papers suggest combining static and dynamic evaluation to balance scalability and contextual understanding [15, 19, 21]. Improving flow-level reasoning and cross-screen context is also highlighted as an important direction [4].

From an accessibility perspective, future research should expand beyond screen reader interaction to address other disability types [1, 19]. Finally, multiple papers emphasize the importance of using those methods as a support for experts rather than to replace them, focusing on clearer explanations and better prioritization of detected issues [1, 15, 17–19, 21].

7 Conclusion

This report analyzed recent research on the use of large language models for automated usability and accessibility testing of mobile applications. The reviewed papers show that LLM-based approaches can effectively support usability evaluation and accessibility auditing, particularly by identifying screen-level usability issues and interaction-dependent accessibility problems that are difficult to capture with traditional rule-based tools.

At the same time, the literature consistently emphasizes that LLM-based methods have clear limitations and do not replace manual testing or expert evaluation. Instead, LLM-based testing is best understood as a complementary tool that can reduce evaluation effort, increase coverage, and support human decision-making when integrated into existing usability and accessibility testing workflows.

References

[1] Wajdi Aljedaani, Ahmed Aljohani, Marcelo M. Eler, Abdulrahman Habib, and Hyunsook Do. 2024. LLMs for Accessibility in Mobile Apps: Detection and Repair. In *Proceedings of the International Conference on Software Engineering (ICSE)*.

[2] Peitong Duan, Jeremy Warner, Yang Li, and Bjoern Hartmann. 2024. Generating Automatic Feedback on UI Mockups with Large Language Models. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[3] Grand View Research. 2024. Mobile Application Market Size, Share & Trends Analysis Report. https://www.grandviewresearch.com/industry-analysis/mobile-application-market Accessed: 2025-02.

[4] Nien-Lin Hsueh, Hsuen-Jen Lin, and Lien-Chi Lai. 2023. Applying Large Language Model to User Experience Testing. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[5] Syed Fatiul Huq, Mahan Tafreshipour, Kate Kalcevich, and Sam Malek. 2023. Reca11: Automated Generation of Accessibility Test Reports from Recorded User Transcripts. In *Proceedings of the International Conference on Software Engineering (ICSE)*.

[6] Interaction Design Foundation. [n. d.]. Accessibility Overview. https://www.interaction-design.org/literature/topics/accessibility Accessed: 2025-02.

[7] Rashedul Islam, Md. Rafiqul Islam, and Tania Mazumder. 2010. Mobile Application and Its Global Impact. *International Journal of Engineering & Technology* (2010).

[8] Guoying Lu, Siyuan Qu, and Yining Chen. 2021. Understanding User Experience for Mobile Applications: A Systematic Literature Review. *International Journal of Human–Computer Interaction* (2021).

[9] Sebastian Lubos, Alexander Felfernig, Gerhard Leitner, and Julian Schwazer. 2024. Towards Recommending Usability Improvements with Multimodal Large Language Models. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[10] MeasuringU. [n. d.]. How Much Does a Usability Test Cost? https://measuringu.com/usability-cost Accessed: 2025-02.

[11] Jakob Nielsen. 1994. Usability Inspection Methods. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[12] Nielsen Norman Group. [n. d.]. Remote Usability-Testing Costs. https://www.nngroup.com/articles/remote-usability-testing-costs/ Accessed: 2025-02.

[13] Ali Ebrahimi Pourasad and Walid Maalej. 2024. Does GenAI Make Usability Testing Obsolete?. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[14] Maryam Taeb, Amanda Swearngin, Eldon Schoop, Ruijia Cheng, Yue Jiang, and Jeffrey Nichols. 2023. AXNav: Replaying Accessibility Tests from Natural Language. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[15] Yuekai Wang and Franceska Xhakaj. 2024. AIHeurEval: Generating Heuristic Evaluations on Multiple UI Screens with Multimodal Large Language Models. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[16] Paweł Weichbroth. 2019. Usability of Mobile Applications: A Consolidated Model. *Journal of System and Software* (2019).

[17] Wei Xiang, Hanfei Zhu, Suqi Lou, Xinli Chen, Zhenghua Pan, Yuping Jin, Shi Chen, and Lingyun Sun. 2023. SimUser: Generating Usability Feedback by Simulating Various Users Interacting with Mobile Applications. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[18] Bo Yang and Shanping Li. 2023. UISGPT: Automated Mobile UI Design Smell Detection with Large Language Models. In *Proceedings of the International Conference on Software Engineering (ICSE)*.

[19] Mingyuan Zhong, Ruolin Chen, Xia Chen, James Fogarty, and Jacob O. Wobbrock. 2024. ScreenAudit: Detecting Screen Reader Accessibility Errors in Mobile Apps Using Large Language Models. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[20] Mingyuan Zhong, Xia Chen, Davin Win Kyi, Chen Li, James Fogarty, and Jacob O. Wobbrock. 2024. TaskAudit: Detecting Accessibility Errors in Mobile Apps via Agentic Task Execution. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.

[21] Ruican Zhong, David W. McDonald, and Gary Hsieh. 2023. Synthetic Heuristic Evaluation: A Comparison between AI- and Human-Powered Usability Evaluation. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.